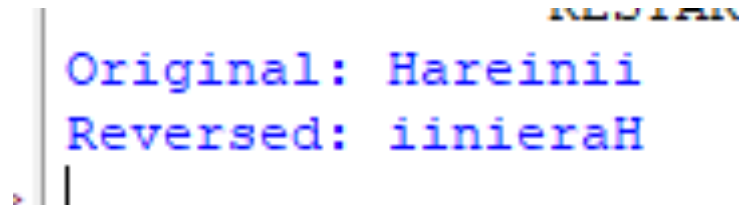


PROGRAM 1 REVERSE USING STACK

CODING:

```
def reverse_string(s):  
    stack = []  
  
    for char in s:  
        stack.append(char)  
  
    reversed_str = ""  
    while stack:  
        reversed_str += stack.pop()  
  
    return reversed_str  
  
input_str = "Hareinii"  
print("Original:", input_str)  
print("Reversed:", reverse_string(input_str))
```

OUTPUT:



```
Original: Hareinii  
Reversed: iinieraH
```

PROGRAM 2: QUEUE OPERATIONS

CODING:

```
class queue:
    def __init__(self):
        self.queue=[]
    def enqueue(self,value,limit):
        self.value=value
        self.limit=limit
        if(len(self.queue)==self.limit):
            print("####OVERFLOW")
        else:
            self.queue.append(self.value)
    def dequeue(self):
        if self.queue==[]:
            print("queue is empty")
        else:
            return self.queue.pop(0)
q=queue()
print("*****WELCOM TO QUEUE OPERATIONS*****")
limit=input("Enter the limit of the queue:")
limit=int(limit)
while True:
    print("1.Enqueue a value")
    print("2.Dequeue a value")
    print("3.View the queue")
    print("4.Exit the queue operation")
    a=input("Enter the required operation:")
    a=int(a)

    if a==1:
        value=input("Enetr the value to be pushed in queue:")
        value=int(value)
        q.enqueue(value,limit)
        print(q.queue)
    elif a==2:
        print("Poped value:",q.dequeue())
    elif a==3:
```

```

        print("Queue=",q.queue)
    elif a==4:
        break
    else:
        print("invalid value")

```

OUTPUT:

```

===== RESIARI: C:\Users\DELL\Desi
*****WELCOM TO QUEUE OPERATIONS*****
Enter the limit of the queue:2
1.Enqueue a value
2.Dequeue a value
3.View the queue
4.Exit the queue operation
Enter the required operation:1
Enetr the value to be pushed in queue:23
[23]
1.Enqueue a value
2.Dequeue a value
3.View the queue
4.Exit the queue operation
Enter the required operation:1
Enetr the value to be pushed in queue:45
[23, 45]
1.Enqueue a value
2.Dequeue a value
3.View the queue
4.Exit the queue operation
Enter the required operation:2
Poped value: 23
1.Enqueue a value
2.Dequeue a value
3.View the queue
4.Exit the queue operation
Enter the required operation:3
Queue= [45]
1.Enqueue a value
2.Dequeue a value
3.View the queue
4.Exit the queue operation
Enter the required operation:4

```

PROGRAM 3: ARRAY 1

CODING :

```
import numpy as np
```

```
array = np.array([1, 3, 5])  
print("creating an array:", array)
```

```
array2 = np.zeros((3, 3))  
print("zero elements:", array2)
```

```
array3 = np.ones((2, 4))  
print("one elements:", array3)
```

```
array1 = np.array([1, 3, 5, 7, 9, 11])  
print("Original array:\n", array1)
```

```
array2 = np.reshape(array1, (2, 3))#2rows 3 col  
print("Reshaped array:", array2)
```

```
array3 = np.transpose(array2)  
print("Transposed array:", array3)
```

```
arr1=np.array([3,24,39])  
arr2=np.array([56,23,78])  
sum_array = arr1 + arr2  
print("Sum Array:", sum_array)
```

OUTPUT:

```
creating an array: [1 3 5]
zero elements: [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
one elements: [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Original array:
 [ 1  3  5  7  9 11]
Reshaped array: [[ 1  3  5]
 [ 7  9 11]]
Transposed array: [[ 1  7]
 [ 3  9]
 [ 5 11]]
Sum Array: [ 59  47 117]
```

PROGRAM 4: ARRAY2

CODING:

```
import numpy as np

array1 = np.array([1, 3, 5, 7, 9, 11])
array2 = np.reshape(array1, (2, 3))

total = np.sum(array1)
mean_val = np.mean(array1)
max_val = np.max(array1)
min_val = np.min(array1)
print("Total:", total, "\nMean:", mean_val, "\nMax:", max_val, "\n Min:", min_val)

slice_part=array2[1:, 2:] # sub-matrix
print("\nSliced Part of Reshaped Matrix:\n", slice_part)

arr_1d=np.array([10, 20, 30])
print("\n1D Array:", arr_1d)

arr_2d=np.array([[1, 2, 3],[4, 5, 6]])
print("\n2D Array:\n", arr_2d)

arr_3d=np.array([[[1, 2, 3],[4, 5, 6]],[[7, 8, 9],[10, 11, 12]]])
print("\n3D Array:\n", arr_3d)

arr_md = np.zeros((2, 2, 2, 2)) # 4D array of zeros
print("\n4D Array:\n", arr_md)
array4 = np.arange(5)#(1,8,2)
print("Using np.arange(5):", array4)
# check the dimension of array1
print("Dimension of array:",array1.ndim)
```

OUTPUT:

```
Total: 36
Mean: 6.0
Max: 11
Min: 1

Sliced Part of Reshaped Matrix:
[[11]]

1D Array: [10 20 30]

2D Array:
[[1 2 3]
 [4 5 6]]

3D Array:
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]

4D Array:
[[[[[0. 0.]
     [0. 0.]]

  [[0. 0.]
   [0. 0.]]]

 [[[[0. 0.]
     [0. 0.]]

  [[0. 0.]
   [0. 0.]]]

 [[[[0. 0.]
     [0. 0.]]

  [[0. 0.]
   [0. 0.]]]

  [[0. 0.]
   [0. 0.]]]]]

Using np.arange(5): [0 1 2 3 4]
Dimension of array: 1
|
```

PROGRAM 6:LINKED LIST

CODING:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new = Node(data)
        if not self.head:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

ll = LinkedList()
ll.insert(1)
ll.insert(2)
ll.insert(3)
ll.display()
```

OUTPUT:


```
1 -> 2 -> 3 -> None
```

PROGRAM 7: LINEAR SEARCH

CODING:

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1  
arr = [2, 3, 4, 10, 40]  
target = 10  
result = linear_search(arr, target)  
if result != -1:  
    print(f"Linear Search: Element found at index {result}")  
else:  
    print("Linear Search: Element not found")
```

OUTPUT:

```
Linear Search: Element found at index 3
```

PROGRAM 8: BINARY SEARCH

CODING:

```
def binary_search(arr, target, low, high):
    if low > high:
        return -1 # Base case: not found

    mid = (low + high) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)

# Example usage
numbers = [3, 8, 15, 22, 27, 31, 42, 56, 64]
target_value = 31

result = binary_search(numbers, target_value, 0, len(numbers) - 1)

if result != -1:
    print(f"Target {target_value} found at index {result}")
else:
    print(f"Target {target_value} not found in the list")
```

OUTPUT:

```
Target 31 found at index 5
|
```

PROGRAM 9: SELECTION SORT

CODING:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

data=input("Enter array to sort:").split(',')
data=list(data)
# Selection Sort Example
data_selection = list(data)
print("\nOriginal list for Selection Sort:", data_selection)
selection_sort(data_selection)
print("Sorted list (Selection Sort):", data_selection)
```

OUTPUT:

```
Enter array to sort:12,33,45,12,23

Original list for Selection Sort: ['12', '33', '45', '12', '23']
Sorted list (Selection Sort): ['12', '12', '23', '33', '45']
```

PROGRAM 10: INSERTION SORT

CODING:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

data=input("Enter array to sort:").split(',')
data=list(data)
# Insertion Sort Example
data_insertion = list(data)
print("\nOriginal list for Insertion Sort:", data_insertion)
insertion_sort(data_insertion)
print("Sorted list (Insertion Sort):", data_insertion)
```

OUTPUT:

```
| Enter array to sort:97,243,12,65,9
|
| Original list for Insertion Sort: ['97', '243', '12', '65', '9']
| Sorted list (Insertion Sort): ['12', '243', '65', '9', '97']
|
```

PROGRAM 11: BUBBLE SORT

CODING:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

data=input("Enter array to sort:").split(',')
data=list(data)
# Bubble Sort Example
data_bubble = list(data)
print("\nOriginal list for Bubble Sort:", data_bubble)
bubble_sort(data_bubble)
print("Sorted list (Bubble Sort):", data_bubble)
```

OUTPUT:

```
Enter array to sort:34,12,45,17,57

Original list for Bubble Sort: ['34', '12', '45', '17', '57']
Sorted list (Bubble Sort): ['12', '17', '34', '45', '57']
```

PROGRAM 12 :TREE TRAVERSAL

CODING:

Define a basic Node structure

class Node:

def __init__(self, value):

self.value = value

self.left = None

self.right = None

Inorder Traversal: Left $\rightarrow$ Root $\rightarrow$ Right

def inorder(node):

if node is None:

return []

return inorder(node.left) + [node.value] + inorder(node.right)

Preorder Traversal: Root $\rightarrow$ Left $\rightarrow$ Right

def preorder(node):

if node is None:

return []

return [node.value] + preorder(node.left) + preorder(node.right)

Postorder Traversal: Left $\rightarrow$ Right $\rightarrow$ Root

def postorder(node):

if node is None:

return []

return postorder(node.left) + postorder(node.right) + [node.value]

A = Node('A')

B = Node('B')

C = Node('C')

D = Node('D')

E = Node('E')

F = Node('F')

```
A.left = B
A.right = C
B.left = D
B.right = E
C.right = F
```

```
print("Inorder:", inorder(A))  # ['D', 'B', 'E', 'A', 'C', 'F']
print("Preorder:", preorder(A)) # ['A', 'B', 'D', 'E', 'C', 'F']
print("Postorder:", postorder(A)) # ['D', 'E', 'B', 'F', 'C', 'A']
```

OUTPUT:

```
Inorder: ['D', 'B', 'E', 'A', 'C', 'F']
Preorder: ['A', 'B', 'D', 'E', 'C', 'F']
Postorder: ['D', 'E', 'B', 'F', 'C', 'A']
|
```


PROGRAM 13: SET

CODING:

```
# Create two sets
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print("Set A:", A)
print("Set B:", B)

# Union
print("\nUnion (A  $\cup$  B):", A.union(B))

# Intersection
print("Intersection (A  $\cap$  B):", A.intersection(B))

# Difference
print("Difference (A - B):", A.difference(B))
print("Difference (B - A):", B.difference(A))

# Symmetric Difference
print("Symmetric Difference (A  $\Delta$  B):", A.symmetric_difference(B))

# Adding element
A.add(10)
print("\nAfter adding :", A)

# Removing element
A.discard(3)
print("After removing:", A)

# Removing duplicates from a list using set
numbers = [1, 2, 2, 3, 4, 4, 5, 6, 6]
unique_numbers = set(numbers)
print("\n\nOriginal List:", numbers)
print("Unique elements using Set:", unique_numbers)
```

OUTPUT:

```
Set A: {1, 2, 3, 4, 5}
Set B: {4, 5, 6, 7, 8}

Union (A  $\cup$  B): {1, 2, 3, 4, 5, 6, 7, 8}
Intersection (A  $\cap$  B): {4, 5}
Difference (A - B): {1, 2, 3}
Difference (B - A): {6, 7, 8}
Symmetric Difference (A  $\Delta$  B): {1, 2, 3, 6, 7, 8}

After adding : {1, 2, 3, 4, 5, 10}
After removing: {1, 2, 4, 5, 10}

Original List: [1, 2, 2, 3, 4, 4, 5, 6, 6]
Unique elements using Set: {1, 2, 3, 4, 5, 6}
|
```

PROGRAM 14: TUPLE

CODING:

```
# 1. Creating a tuple
tup = (10, 20, 30, 40, 50)
print("Tuple:", tup)

# Access elements
print("First element:", tup[0])
print("Last element:", tup[-1])

# Slice
print("Slice tup[1:4]:", tup[1:4])

# 2. Concatenate tuples
t1 = (1, 2, 3)
t2 = (4, 5, 6)
t3 = t1 + t2
print("\nConcatenated Tuple:", t3)

# 3. Max, Min, Length
print("Maximum:", max(t3))
print("Minimum:", min(t3))
print("Length:", len(t3))
```

OUTPUT:

```
Tuple: (10, 20, 30, 40, 50)
First element: 10
Last element: 50
Slice tup[1:4]: (20, 30, 40)

Concatenated Tuple: (1, 2, 3, 4, 5, 6)
Maximum: 6
Minimum: 1
Length: 6
```

PROGRAM 15 : DICTIONARY

CODING:

```
# Initialize dictionary
student = {
    "name": "Arun",
    "age": 20,
    "marks": 85
}

# Accessing values safely
print("Name:", student.get("name"))      # Arun
print("City:", student.get("city", "N/A")) # N/A

# Adding/updating values
student.update({"city": "Coimbatore", "marks": 90})

# Display all keys and values
print("Keys:", list(student.keys()))
print("Values:", list(student.values()))
print("Items:", list(student.items()))

# Remove a key
removed = student.pop("age")
print("Removed age:", removed)

# Set default value
student.setdefault("gender", "Male")

# Copy dictionary
backup = student.copy()

# Clear dictionary
student.clear()

print("\nBackup copy:", backup)
print("Original after clear:", student)
```

OUTPUT:

```
Name: Arun
City: N/A
Keys: ['name', 'age', 'marks', 'city']
Values: ['Arun', 20, 90, 'Coimbatore']
Items: [('name', 'Arun'), ('age', 20), ('marks', 90), ('city', 'Coimbatore')]
Removed age: 20

Backup copy: {'name': 'Arun', 'marks': 90, 'city': 'Coimbatore', 'gender': 'Male'}
Original after clear: {}
|
```